DOCUMENTACIÓN DEL PROYECTO

Sistema de Gestión de Hospital Universitario

Desarrollado en Java con Programación Orientada a Objetos

Asignatura: Programación I

Integrantes:

Jafeth Guevara

Jasson Samudio

Luis Zhong

Dilan González

Fecha de entrega: 8/11/26

Introducción

Este documento presenta el Sistema de Gestión de Hospital Universitario, desarrollado en Java con Programación Orientada a Objetos (POO). El sistema permite gestionar pacientes, médicos, citas médicas, tratamientos, historiales clínicos, recetas, medicamentos, farmacia e inventario.

El proyecto se organizó en clases que representan los elementos del hospital, y en clases de gestión que controlan los registros y aplican reglas. El programa cuenta con un menú principal para usar sus funciones.

Objetivo general

Desarrollar un sistema en Java, aplicando Programación Orientada a Objetos, que permita gestionar pacientes, médicos, citas, medicamentos e inventario de un hospital universitario.

Objetivos específicos

1. Diseñar las clases que representan los elementos principales del hospital.
2. Implementar clases de gestión para registrar y consultar pacientes y médicos, y para agendar citas.
3. Evitar IDs y cédulas repetidos, y citas en la misma fecha y hora para un mismo médico.
4. Relacionar a los médicos con sus especialidades y departamentos, y mostrar los médicos según el motivo de consulta.
5. Controlar los medicamentos y sus cantidades mediante un inventario relacionado con la farmacia.
6. Registrar acciones del sistema mediante la función de auditoría de la clase Administrador.

Descripción general del sistema

El sistema se maneja desde un menú principal, donde se accede al registro y consulta de pacientes y médicos, y al agendamiento de citas. Al agendar, el sistema puede mostrar los médicos de una especialidad y los horarios disponibles de un médico según las citas registradas. Un médico no puede tener dos citas en la misma fecha y hora.

En la parte de farmacia, el sistema maneja los medicamentos y sus cantidades en un inventario relacionado con la farmacia. La clase Administrador registra acciones en su lista de auditoría. [Completar: nombre de la clase principal y opciones exactas del menú, que no estaban en los archivos revisados]

Tecnologías y herramientas utilizadas

- Lenguaje Java: lenguaje con el que se desarrolló el sistema. [Completar: versión del JDK]
- Programación Orientada a Objetos: paradigma usado para organizar el sistema en clases y objetos.
- Librerías estándar de Java: java.util (ArrayList, List y Collections) para las listas, y java.time (LocalDate, LocalTime y DayOfWeek) para validar fechas y horas.
- Entorno de desarrollo: [Completar: IDE utilizado]

Programación orientada a objetos aplicada en el proyecto

La POO organiza un programa en clases y objetos. Estos son los conceptos aplicados en el proyecto:

- Clases y objetos: una clase es un molde y un objeto es un elemento creado a partir de ella. Paciente, Medico y Cita son clases; un paciente registrado por el usuario es un objeto de Paciente.
- Abstracción: cada clase guarda solo la información necesaria para su función en el hospital.
- Encapsulamiento: todos los atributos son privados y se acceden mediante getters y setters. Los setters validan los datos, y las listas se devuelven como copias o como listas de solo lectura.
- Asociación: una clase usa o contiene a otra, como la cita con el paciente y el médico, o la farmacia con su inventario.
- Separación de responsabilidades: GestionPacientes, GestionMedicos y GestionCitas registran, consultan y validan; las demás clases representan los datos.
- Sobrescritura: todas las clases, excepto las de gestión, sobrescriben el método toString() para mostrar su información. Las clases del proyecto no usan herencia entre ellas.

Descripción de las clases implementadas

Cada clase se describe por su función. Sus atributos y métodos se detallan en la Tabla 2.

Paciente: Representa a la persona que recibe atención. Guarda su ID, nombre, apellido y cédula, y valida que el nombre, el apellido y la cédula no estén vacíos.

Medico: Representa al profesional que atiende a los pacientes. Guarda su ID, nombre y apellido, y está relacionado con una Especialidad y un Departamento, que son obligatorios.

Cita: Representa una cita médica. Relaciona a un Paciente con un Medico en una fecha (formato AAAA-MM-DD) y una hora (formato HH:MM), que valida al crearla. Su método esMismaFechaHora compara la fecha y la hora de dos citas.

Tratamiento: Representa un tratamiento indicado a un paciente por un médico. Guarda una descripción y una duración, además del Paciente y el Medico relacionados.

Historial: Representa el historial clínico de un paciente. Guarda una lista de sus citas y una lista de sus tratamientos, y evita agregar elementos repetidos.

Receta: Representa una receta emitida por un médico para un paciente. Guarda una lista de medicamentos a los que se puede agregar o quitar elementos.

Medicamento: Representa un medicamento con su ID, nombre, descripción y cantidad. Permite aumentar y disminuir la cantidad, y comprobar si hay existencias.

Especialidad: Representa el área de la medicina de un médico. Guarda su ID, nombre y descripción.

Departamento: Representa el departamento del hospital al que pertenece un médico. Guarda su ID, nombre y descripción.

Administrador: Representa al usuario administrador. Guarda su ID, nombre, usuario y rol, valida su rol y registra acciones en una lista de auditoría.

Farmacia: Representa la farmacia del hospital. Guarda su ID, nombre y ubicación, y se le puede asignar un Inventario.

Inventario: Controla los medicamentos de una farmacia. Permite agregar medicamentos sin repetirlos, buscarlos por ID, retirar cantidades y mostrarlos.

GestionPacientes: Clase de gestión de pacientes. Registra pacientes evitando IDs y cédulas repetidas, los busca por ID o por cédula y los muestra.

GestionMedicos: Clase de gestión de médicos. Registra médicos evitando IDs repetidos, los busca por ID o por nombre de especialidad y los muestra.

GestionCitas: Clase de gestión de citas. Registra citas evitando IDs repetidos y que un médico tenga dos citas en la misma fecha y hora, las busca por ID, paciente o médico, calcula los horarios disponibles y las muestra.

Tabla 1: Escenarios del sistema y clases que participan

Tabla 1. Escenarios del sistema y clases participantes

N.º	Escenario	Clases que participan	Participación en el escenario
1	Registrar un paciente sin repetir su ID ni su cédula	Paciente GestionPacientes	GestionPacientes comprueba que el ID y la cédula no existan antes de guardar el objeto Paciente.
2	Consultar pacientes registrados	GestionPacientes Paciente	GestionPacientes busca pacientes por ID o cédula, devuelve la lista y los muestra.
3	Registrar un médico y relacionarlo con su especialidad y departamento	Medico Especialidad Departamento GestionMedicos	El médico se crea con una Especialidad y un Departamento obligatorios; GestionMedicos evita IDs repetidos.
4	Consultar médicos registrados	GestionMedicos Medico	GestionMedicos busca por ID, devuelve la lista y muestra los médicos.
5	Mostrar los médicos de una especialidad	GestionMedicos Medico Especialidad	GestionMedicos busca los médicos cuya especialidad coincide con el nombre indicado.
6	Agendar una cita médica	Cita Paciente Medico GestionCitas	GestionCitas registra la cita que une al paciente con el médico y evita IDs repetidos.
7	Evitar que un médico tenga dos citas en la misma fecha y hora	GestionCitas Cita Medico	GestionCitas compara las citas del mismo médico con el método esMismaFechaHora de Cita.
8	Mostrar horarios disponibles	GestionCitas Cita Medico	GestionCitas revisa los horarios de 08:00 a 15:00 y descarta los ocupados por citas del médico en esa fecha.
9	Manejar medicamentos y sus cantidades en el inventario	Medicamento Inventario	Inventario agrega, busca, retira y muestra medicamentos; Medicamento controla su cantidad.
10	Relacionar la farmacia con su inventario	Farmacia Inventario	Farmacia guarda su Inventario mediante asignarInventario; Inventario guarda su Farmacia.
11	Registrar acciones del sistema (auditoría)	Administrador	Administrador guarda el texto de cada acción en su lista de auditoría mediante registrarAuditoria.
12	Manejar tratamientos e historial clínico	Tratamiento Historial Cita Paciente Medico	Historial guarda las citas y los tratamientos de un paciente; Tratamiento relaciona a un paciente con un médico.
13	Manejar recetas	Receta Medicamento Paciente Medico	Receta guarda los medicamentos indicados por un médico a un paciente.

Tabla 2: Clases, atributos y métodos implementados

La Tabla 2 muestra los atributos y métodos de las 15 clases, tomados del código fuente. Todos los atributos son privados y las clases están en el paquete utp.Ac.Pa.sistema.domain. Las clases que no tienen setters no los incluyen.

Tabla 2. Clases, atributos y métodos implementados

Clase	Atributos	Métodos
Paciente	int id String nombre String apellido String cedula	Constructor: Paciente(int id, String nombre, String apellido, String cedula) Getters: getId(), getNombre(), getApellido(), getCedula() Setters: setNombre(String), setApellido(String), setCedula(String) Otros: toString()
Medico	int id String nombre String apellido Especialidad especialidad Departamento departamento	Constructor: Medico(int id, String nombre, String apellido, Especialidad especialidad, Departamento departamento) Getters: getId(), getNombre(), getApellido(), getEspecialidad(), getDepartamento() Setters: setNombre(String), setApellido(String), setEspecialidad(Especialidad), setDepartamento(Departamento) Otros: toString()
Cita	int id Paciente paciente Medico medico String fecha String hora	Constructor: Cita(int id, Paciente paciente, Medico medico, String fecha, String hora) Getters: getId(), getPaciente(), getMedico(), getFecha(), getHora() Setters: setFecha(String), setHora(String) Otros: boolean esMismaFechaHora(Cita otraCita), toString()
Tratamiento	int id String descripcion String duracion Paciente paciente Medico medico	Constructor: Tratamiento(int id, String descripcion, String duracion, Paciente paciente, Medico medico) Getters: getId(), getDescripcion(), getDuracion(), getPaciente(), getMedico() Setters: setDescripcion(String), setDuracion(String) Otros: toString()
Historial	int id Paciente paciente List<Cita> citas List<Tratamiento> tratamientos	Constructor: Historial(int id, Paciente paciente) Getters: getId(), getPaciente(), getCitas(), getTratamientos() Otros: void agregarCita(Cita cita), void agregarTratamiento(Tratamiento tratamiento), toString()
Receta	int id Paciente paciente Medico medico List<Medicamento> medicamentos	Constructor: Receta(int id, Paciente paciente, Medico medico) Getters: getId(), getPaciente(), getMedico(), getMedicamentos() Otros: void agregarMedicamento(Medicamento medicamento), void eliminarMedicamento(Medicamento medicamento), toString()

Clase	Atributos	Métodos
Medicamento	int id String nombre String descripcion int cantidad	Constructor: Medicamento(int id, String nombre, String descripcion, int cantidad) Getters: getId(), getNombre(), getDescripcion(), getCantidad() Setters: setNombre(String), setDescripcion(String) Otros: void aumentarCantidad(int cantidad), boolean disminuirCantidad(int cantidad), boolean hayExistencias(), toString()
Especialidad	int id String nombre String descripcion	Constructor: Especialidad(int id, String nombre, String descripcion) Getters: getId(), getNombre(), getDescripcion() Setters: setNombre(String), setDescripcion(String) Otros: toString()
Departamento	int id String nombre String descripcion	Constructor: Departamento(int id, String nombre, String descripcion) Getters: getId(), getNombre(), getDescripcion() Setters: setNombre(String), setDescripcion(String) Otros: toString()
Administrador	int id String nombre String usuario String rol List<String> auditoria	Constructor: Administrador(int id, String nombre, String usuario, String rol) Getters: getId(), getNombre(), getUsuario(), getRol(), getAuditoria() Setters: setNombre(String), setUsuario(String), setRol(String) Otros: boolean validarRol(String rolRequerido), void registrarAuditoria(String accion), toString()
Farmacia	int id String nombre String ubicacion Inventario inventario	Constructor: Farmacia(int id, String nombre, String ubicacion) Getters: getId(), getNombre(), getUbicacion(), getInventario() Setters: setNombre(String), setUbicacion(String) Otros: void asignarInventario(Inventario inventario), toString()
Inventario	int id Farmacia farmacia List<Medicamento> medicamentos	Constructor: Inventario(int id, Farmacia farmacia) Getters: getId(), getFarmacia(), getMedicamentos() Otros: void agregarMedicamento(Medicamento medicamento), Medicamento buscarMedicamento(int id), boolean retirarMedicamento(int id, int cantidad), void mostrarMedicamentos(), toString()
GestionPacientes	List<Paciente> pacientes	Constructor: GestionPacientes() Getters: getPacientes() Otros: void registrarPaciente(Paciente paciente), Paciente buscarPorId(int id), Paciente buscarPorCedula(String cedula), void mostrarPacientes()
GestionMedicos	List<Medico> medicos	Constructor: GestionMedicos() Getters: getMedicos() Otros: void registrarMedico(Medico medico), Medico buscarPorId(int id), List<Medico> buscarPorEspecialidad(String nombreEspecialidad), void mostrarMedicos()

Clase	Atributos	Métodos
GestionCitas	List<Cita> citas	Constructor: GestionCitas() Getters: getCitas() Otros: void registrarCita(Cita cita), List<String> obtenerHorariosDisponibles(int medicoId, String fecha), Cita buscarPorId(int id), List<Cita> buscarPorPaciente(int pacienteId), List<Cita> buscarPorMedico(int medicoId), void mostrarCitas()

Descripción de las principales funcionalidades

1. Registro de pacientes: evita que se repitan el ID y la cédula.
2. Consulta de pacientes registrados.
3. Consulta de médicos, relacionados con su especialidad y su departamento.
4. Búsqueda de médicos por especialidad, para mostrar los médicos que corresponden a una consulta. [Confirmar en la clase principal cómo el menú pide el motivo de consulta]
5. Agendamiento de citas médicas entre un paciente y un médico, con fecha y hora.
6. Control de citas duplicadas: un médico no puede tener dos citas en la misma fecha y hora.
7. Horarios disponibles: de lunes a viernes, en horas exactas de 08:00 a 15:00, se descartan los ocupados por citas del médico y no se aceptan fechas pasadas.
8. Manejo de medicamentos y sus cantidades en el inventario, relacionado con la farmacia.
9. Auditoría: la clase Administrador guarda el texto de cada acción registrada y valida su rol.
10. Tratamientos, historiales y recetas: Tratamiento relaciona un paciente con un médico; Historial guarda las citas y tratamientos de un paciente; Receta guarda y quita medicamentos. [Confirmar si el menú principal ofrece estas opciones]

Conclusión

El sistema permitió aplicar los conceptos de la Programación Orientada a Objetos en Java. Organiza la información del hospital en clases y usa clases de gestión que evitan IDs y cédulas repetidos, y citas duplicadas. También relaciona a los médicos con sus especialidades y departamentos, muestra horarios disponibles, maneja medicamentos mediante un inventario relacionado con la farmacia y registra acciones con la auditoría del Administrador.

[Completar: aprendizajes del equipo y comentarios finales]